

Reading the numbers in Supabase

A no-experience-needed guide to logging in, running the eight Foundation Clinic reporting queries, and reading what comes back — even if you've never touched SQL before.

Foundation Clinic · prepared September 2026

Everything in this guide is **read-only reporting**. You're pulling numbers out to look at them — you are not changing any data. Running these queries as many times as you like is completely safe. If something looks off, you can't break anything; just ask.

What's in here

- | | |
|--|-----------------------------------|
| 1 Logging in to Supabase | 4 Reading & exporting the results |
| 2 Finding the SQL Editor | 5 The eight queries, one by one |
| 3 Running a query (the one shortcut to remember) | 6 Golden rules & common hiccups |

1 · Logging in to Supabase

Supabase is the web tool that stores all of the clinic's numbers. You reach it in a browser — there's nothing to install.

- 1 Open the invite email from Supabase and click **Accept / Join**. This links your account to the Foundation Clinic workspace. (Do this once; after that you just log in normally.)
- 2 Go to supabase.com/dashboard and sign in. Use whichever sign-in the invite set up — usually your Google account or an email + password you chose when accepting.
- 3 On the dashboard you'll see the **Foundation Clinic** project. Click it to open it.

Bookmark it. Once you're in, save the page in your browser so you can come straight back next time.

2 · Finding the SQL Editor

The queries all run in one place, called the **SQL Editor**.

- 1 With the project open, look at the icons down the **left-hand edge** of the screen.
- 2 Click the one that looks like a small terminal / **SQL** icon — hovering shows the label **SQL Editor**.
- 3 You'll get a big empty text box. That's where a query goes. Click **+ New query** (top area) if you want a fresh blank one.

Think of it as a search box for the database: you paste in a question written in SQL, run it, and a table of answers appears underneath.

3 · Running a query — the one shortcut to remember

- 1 **Copy** one of the queries from Section 5 and **paste** it into the empty SQL Editor box.
- 2 Press **Cmd** + **Enter** to run it. (On a Windows keyboard that's **Ctrl** + **Enter**.) You can also click the green **Run** button in the bottom-right.
- 3 A results table appears below within a second or two. Done.

This is the #1 thing people miss: pressing plain **Enter** just adds a new line — it does *not* run anything. You must press **Cmd** + **Enter** (or click **Run**).

One query at a time. If you paste a query over an old one, select all the old text first (**Cmd** + **A**) and delete it, so only the new query is in the box.

4 · Reading & exporting the results

Results come back as a table. Each **row** is usually one month; each **column** is one number. Most queries here are sorted oldest month at the top, newest at the bottom.

- Money columns are in **euros**. Percentages are already turned into percentages (e.g. **18.5** means 18.5%).
- A blank / **null** cell means "no value for that month" — normal for early months.
- To pull the numbers into Excel or Google Sheets, click **Export** (or the download icon above the results) and choose **CSV**.

The dashboard and these queries always agree. Each query below is the exact same calculation the live dashboard uses — so a number here will match the matching dashboard chart. If they ever disagree, that's worth flagging.

5 · The eight queries, one by one

For six of these, the whole query is a single short line — the heavy lifting lives inside a saved "view" in the database, so you just ask for it. Two of them (active customers, churn) are longer blocks; paste the whole thing.

1 · Member growth by month

view: `v_member_tiers_by_month` · matches the Members chart

```
SELECT * FROM v_member_tiers_by_month ORDER BY month;
```

What you get: one row per month with the member count in each tier (t1–t4, custom), the legacy pre-Feb-2026 members, the `total`, and `growth_pct` — how much the total grew or shrank versus the month before.

2 · GTV run rate (annualised)

views: `v_gtv_run_rate` (latest) · `v_gtv_run_rate_by_month` (every month)

```
SELECT * FROM v_gtv_run_rate;

-- or, the full monthly series:
SELECT * FROM v_gtv_run_rate_by_month ORDER BY month;
```

What you get: the annualised run rate — the average GTV of the last three closed months, multiplied by 12. It's a "if the recent pace held for a year" figure. The first query gives the single latest number; the second shows it for every month.

3 · Net revenue by month

view: `v_net_revenue_by_month` · matches the Net-revenue card

```
SELECT * FROM v_net_revenue_by_month ORDER BY month;
```

What you get: net revenue per month (memberships + clinical incl. partner margin + product sales, excluding through-billing pass-through). The `provisional` column is `true` for the current still-open month.

4 · MRR & ARR by month

view: `v_mrr_by_month` · matches the MRR/ARR cards

```
SELECT * FROM v_mrr_by_month ORDER BY month;
```

What you get: monthly recurring revenue (`mrr`) as a run-rate snapshot at month-end, and `arr` which is simply $MRR \times 12$. These are *non-prorated* — a member counts at their full monthly fee regardless of what day they joined.

5 · Active customers by month

longer query — paste the whole block

```
WITH months AS (  
  SELECT gs::date AS m_start,  
         (gs + interval '1 month - 1 day')::date AS m_end,  
         to_char(gs,'YYYY-MM') AS ym  
  FROM generate_series(  
    date_trunc('month', (SELECT min(valid_from) FROM memberships WHERE tier <>  
'project')),  
    date_trunc('month', CURRENT_DATE),  
    interval '1 month') gs  
,  
members AS (  
  SELECT mo.ym, count(DISTINCT me.patient_id) AS active_members  
  FROM months mo  
  JOIN memberships me  
    ON me.tier <> 'project'  
   AND me.valid_from <= mo.m_end  
   AND (me.valid_to IS NULL OR me.valid_to >= mo.m_start)  
  GROUP BY mo.ym  
,  
walkins AS (  
  SELECT mo.ym, count(DISTINCT v.patient_id) AS walkins_served  
  FROM months mo  
  JOIN ajas_visits v ON v.service_month = mo.ym  
   AND (v.onboarding_tagged IS TRUE OR COALESCE(v.services_ex_vat,0) > 0)  
  WHERE NOT EXISTS (  
    SELECT 1 FROM memberships me  
    WHERE me.patient_id = v.patient_id AND me.tier <> 'project'  
      AND me.valid_from <= mo.m_end  
      AND (me.valid_to IS NULL OR me.valid_to >= mo.m_start))  
  GROUP BY mo.ym  
)  
SELECT mo.ym AS month,  
       COALESCE(m.active_members,0) AS active_members,  
       COALESCE(w.walkins_served,0) AS walkins_served,  
       COALESCE(m.active_members,0) + COALESCE(w.walkins_served,0) AS active_customers  
FROM months mo  
LEFT JOIN members m ON m.ym = mo.ym  
LEFT JOIN walkins w ON w.ym = mo.ym  
ORDER BY mo.ym;
```

What you get: per month, the number of active members, the non-members served that month (walk-ins / onboardings), and the two added together as `active_customers`. Nobody is counted twice.

6 · Member churn by month

longer query — paste the whole block · matches the Churn card

```
WITH lm AS (  
  SELECT patient_id, bool_or(valid_to IS NULL) AS has_open, max(valid_to) AS last_end  
  FROM memberships WHERE tier <> 'project' GROUP BY patient_id  
) ,  
churn AS (  
  SELECT date_trunc('month', last_end)::date AS churn_month, count(*) AS n  
  FROM lm WHERE has_open = false AND last_end IS NOT NULL GROUP BY 1  
) ,  
bounds AS (  
  SELECT date_trunc('month', (SELECT min(valid_from) FROM memberships WHERE  
tier<>'project'))::date AS s ,  
    date_trunc('month', CURRENT_DATE)::date AS e  
) ,  
months AS (  
  SELECT gs::date AS m_start, (gs + interval '1 month - 1 day')::date AS m_end ,  
to_char(gs, 'YYYY-MM') AS ym  
  FROM bounds, generate_series((SELECT s FROM bounds), (SELECT e  
FROM bounds), interval '1 month') gs  
) ,  
m AS (  
  SELECT mo.*, COALESCE(c.n,0) AS churned_m ,  
    SUM(COALESCE(c.n,0)) OVER (ORDER BY mo.m_start ROWS BETWEEN 5 PRECEDING AND  
CURRENT ROW) AS churned_6mo ,  
    SUM(COALESCE(c.n,0)) OVER (ORDER BY  
mo.m_start) AS churned_cum  
  FROM months mo LEFT JOIN churn c ON c.churn_month = mo.m_start  
)  
SELECT m.ym AS month ,  
  m.churned_m , b0.members AS base_month_start , round(100.0*m.churned_m /  
NULLIF(b0.members,0),1) AS churn_pct_month ,  
  m.churned_6mo,b6.members AS base_6mo_start , round(100.0*m.churned_6mo/  
NULLIF(b6.members,0),1) AS churn_pct_6mo ,  
  m.churned_cum,bv.members AS members_ever_todate,round(100.0*m.churned_cum/  
NULLIF(bv.members,0),1) AS churn_pct_cumulative  
FROM m  
CROSS JOIN LATERAL (SELECT count(DISTINCT me.patient_id) AS members FROM memberships  
me  
  WHERE me.tier<>'project' AND me.valid_from <= m.m_start AND (me.valid_to IS NULL OR  
me.valid_to >= m.m_start)) b0  
CROSS JOIN LATERAL (SELECT count(DISTINCT me.patient_id) AS members FROM memberships  
me  
  WHERE me.tier<>'project' AND me.valid_from <= (m.m_start - interval '5  
months')::date  
  AND (me.valid_to IS NULL OR me.valid_to >= (m.m_start - interval '5  
months')::date)) b6  
CROSS JOIN LATERAL (SELECT count(DISTINCT me.patient_id) AS members FROM memberships  
me
```

```
WHERE me.tier<>'project' AND me.valid_from <= m.m_end) bev
ORDER BY m.ym;
```

What you get: for each month, how many members left (`churned_m`), plus a steadier trailing-6-month rate (`churn_pct_6mo`) and a cumulative rate. On a small book the monthly number is often 0 — the **6-month rate is the one to watch**.

7 · GTV by month

view: `v_gtv_by_month` · matches the GTV chart

```
SELECT * FROM v_gtv_by_month ORDER BY month;
```

What you get: total Gross Transaction Value per month, broken into membership & availability, clinical (incl. partner margin), and through-billing, with the `gtv` total. The `provisional` column flags the current open month.

8 · Gross margin by month

view: `v_gross_margin_by_month` · matches the CM chart

```
SELECT * FROM v_gross_margin_by_month ORDER BY month;
```

What you get: revenue, cost and the resulting gross margin per month. Divide margin by revenue for the % (the dashboard shows the %). The current open month appears with `provisional = true` .

6 · Golden rules & common hiccups

SITUATION	WHAT TO DO
Nothing happens when I press Enter	Press Cmd + Enter (or click Run). Plain Enter only makes a new line.
Red error message appears	Usually leftover text from a previous query got mixed in. Select all (Cmd + A), delete, paste just one query, run again.
"relation ... does not exist"	The view name was mistyped or partly deleted. Re-copy the whole line from this guide.
Numbers look wrong / don't match the dashboard	Don't edit the number — flag it. These queries are built to match the dashboard exactly, so a mismatch is worth reporting.
I want these in a spreadsheet	Run the query, then click Export → CSV above the results and open it in Excel or Sheets.
Did I break something?	No. Every query here only <i>reads</i> data. You can run them as often as you like.

The only rule that matters: paste one query, press **Cmd** + **Enter**, read the table. That's the whole job. When in doubt, ask — nothing you do here can harm the data.